BACKEND PROGRESS REPORT

AI-Based Smart Learning Assistant

Server-Side Development, Database & Authentication — Week 5

CSE4104-7A-T01
Project Type: Backend

Section 7A · Team 01
Jun 2026

Status: In Progress
v1.0

1. PROJECT & TEAM INFORMATION

Project Title	AI-Based Smart Learning Assistant
Course Code	CSE4104-7A-T01
Team Section	7A — Team 01
Submission Week	Week 5 — Backend Development & Database Implementation
Repository	github.com/<org>/smart-learning-assistant (update with live link before submission)
Report Date	June 2026

2. BACKEND TECHNOLOGY STACK

The backend was implemented using the following stack. All components run locally without any external paid services, enabling full reproducibility for grading.

Runtime	Node.js v22.22.2 (LTS)
Framework	Express.js v4.19 — REST API routing and middleware
Database	SQLite (via Node.js built-in node:sqlite module) — see Section 3 for production note
Authentication	JWT (jsonwebtoken) + bcryptjs password hashing (salt factor 12)
Validation	express-validator — request body/param validation
Security	helmet (HTTP headers), cors, express-rate-limit (200 req/15min global, 20 req/15min on auth)
Logging	morgan (HTTP request logging, dev format)
AI Integration	Service-layer abstraction (src/services/ai.service.js) — OpenAI GPT-4 compatible interface

Note on database engine: the schema, all relations, and every query in this submission are written to be 100% portable to PostgreSQL — the production target specified in the Week 4 System Architecture Diagram. SQLite was used for this local development and grading environment because it requires no separate database server process, making the project runnable with a single "npm install && npm run dev" command on any grader's machine.

3. DATABASE DESIGN SUMMARY

The database implements all 11 entities from the ER Diagram submitted in Week 4, with primary keys, foreign keys, and CHECK constraints enforcing valid enum values. Foreign key enforcement is enabled via PRAGMA foreign_keys = ON.

Table	Rows	Primary / Foreign Keys	Purpose
`users`	2	id (PK)	Student/admin accounts, auth, streak tracking
`refresh_tokens`	4	id (PK), user_id (FK)	JWT refresh token storage with expiry

subjects	**5**	id (PK)	Subject catalog (Math, Physics, CS, etc.)
user_subjects	**3**	id (PK), user_id+subject_id (FK)	M:N — user proficiency per subject
tasks	**3**	id (PK), user_id+subject_id (FK)	Task management — status, priority, due dates
study_plans	**0**	id (PK), user_id (FK)	AI-generated and manual study plans
chat_sessions	**1**	id (PK), user_id (FK)	AI chatbot conversation sessions
chat_messages	**2**	id (PK), session_id (FK)	Individual chat messages (user + assistant)
recommendations	**0**	id (PK), user_id+subject_id (FK)	AI-curated learning resources
study_logs	**1**	id (PK), user_id+subject_id (FK)	Logged study sessions for analytics
notifications	**0**	id (PK), user_id (FK)	In-app notifications

Full schema source: database/schema.sql — see Section 8 (Screenshots) for live database inspection output.

4. IMPLEMENTED API ENDPOINTS

All endpoints below are implemented, running, and verified via manual API testing (see Section 7).

4.1 Authentication APIs

POST	`/api/auth/register`	Register a new user (name, email, password) — returns JWT access token
POST	`/api/auth/login`	Authenticate with email/password — returns JWT access token + refresh cookie
POST	`/api/auth/logout`	Invalidate refresh token and clear session cookie
POST	`/api/auth/refresh`	Exchange a valid refresh token for a new access token
GET	`/api/auth/me`	Return the currently authenticated user's profile

4.2 User Profile APIs

GET	`/api/users/profile`	Get full profile including enrolled subjects
PUT	`/api/users/profile`	Update name, bio, timezone, study goal, avatar
PUT	`/api/users/password`	Change password (requires current password)

4.3 Task Management APIs

GET	`/api/tasks`	List tasks — filterable by status, priority, subjectId, search
POST	`/api/tasks`	Create a new task
GET	`/api/tasks/:id`	Get single task by ID
PUT	`/api/tasks/:id`	Update task fields
PATCH	`/api/tasks/:id/status`	Update task status (TODO/IN_PROGRESS/DONE/CANCELLED)
DELETE	`/api/tasks/:id`	Delete a task
GET	`/api/tasks/stats`	Aggregate counts — total, done, in-progress, overdue

4.4 AI Chat APIs

GET	`/api/chat/sessions`	List all chat sessions for the user
POST	`/api/chat/sessions`	Create a new chat session
GET	`/api/chat/sessions/:id`	Get a session with full message history
POST	`/api/chat/sessions/:id/messages`	Send a message — saves it, calls AI service, saves & returns reply
DELETE	`/api/chat/sessions/:id`	Delete a chat session and its messages

4.5 Dashboard & Subjects APIs

GET	`/api/dashboard`	Task summary, weekly study hours, streak, upcoming tasks

POST	`/api/dashboard/study-log`	Log a manual study session
GET	`/api/subjects`	List all subjects in the catalog
POST	`/api/subjects`	Create a new subject (admin use)
POST	`/api/subjects/user`	Enroll current user in a subject with proficiency

5. AUTHENTICATION WORKFLOW

The system implements JWT-based stateless authentication with refresh token rotation, following the minimum requirements specified in the assignment brief.

5.1 Registration / Login Flow

- Client submits credentials → POST /api/auth/register or /api/auth/login
- express-validator checks email format and required fields before reaching the controller
- Password is hashed with bcryptjs (salt factor 12) before storage — plaintext passwords are never persisted
- On login, bcrypt.compare() verifies the submitted password against the stored hash
- On success, two tokens are issued: a short-lived JWT access token (15 min) and a longer-lived refresh token (7 days)
- The refresh token is persisted in the refresh_tokens table and also set as an httpOnly cookie; the access token is returned in the JSON response body for the client to attach to subsequent requests

5.2 Protected Route Access

- Every protected route is wrapped with the authenticate middleware (src/middleware/auth.middleware.js)
- The middleware extracts the Bearer token from the Authorization header and verifies its signature with jwt.verify()
- On success, req.user is populated with { id, email, role } and the request proceeds to the controller
- On failure (missing, malformed, or expired token), the middleware immediately returns 401 Unauthorized — no database access occurs for invalid requests

5.3 Role Management

The users.role column supports STUDENT (default) and ADMIN values. A requireAdmin middleware is implemented and ready to guard future admin-only routes (e.g., subject catalog management, user moderation) as the project's admin panel is built out in later weeks.

6. ERROR HANDLING

A centralized error-handling middleware (src/middleware/error.middleware.js) catches all thrown and forwarded errors so the server never crashes on bad input.

Scenario	HTTP Status	Handling Strategy
Invalid input (missing/malformed fields)	422	express-validator middleware returns a structured { field, message } array before the controller runs
Duplicate email on register	409	Explicit check against users table; returns a clear conflict message
Invalid login credentials	401	Generic "Invalid credentials" message — avoids leaking whether the email exists
Missing/expired/ invalid JWT	401	auth.middleware.js rejects immediately, before any DB query runs
Resource not found (task, session, etc.)	404	Each controller checks ownership + existence before mutating
Unhandled exceptions	500	Caught by the global error handler; stack trace logged server-side only, generic message returned to client in production
Unknown route	404	Catch-all handler returns a consistent JSON 404 instead of an HTML

		error page

7. API TESTING RESULTS

20 manual test requests were executed against the running local server (http://localhost:4000) covering success paths, validation failures, authentication failures, and not-found scenarios. All requests returned the expected status code.

#	Test Case	Endpoint	Result
1	Register duplicate email → conflict	`POST /auth/register`	**409 ✓**
2	Register new user → success	`POST /auth/register`	**201 ✓**
3	Register with weak password → validation error	`POST /auth/register`	**422 ✓**
4	Login with correct credentials	`POST /auth/login`	**200 ✓**
5	Login with wrong password → rejected	`POST /auth/login`	**401 ✓**
6	Access protected route with no token	`GET /auth/me`	**401 ✓**
7	Access protected route with valid token	`GET /auth/me`	**200 ✓**
8	Create a new task	`POST /tasks`	**201 ✓**
9	List tasks for authenticated user	`GET /tasks`	**200 ✓**
10	Update task status to DONE	`PATCH /tasks/:id/status`	**200 ✓**
11	Get task statistics	`GET /tasks/stats`	**200 ✓**
12	Create a new chat session	`POST /chat/sessions`	**201 ✓**
13	Send message → AI service responds	`POST /chat/sessions/:id/messages`	**200 ✓**
14	Retrieve session with message history	`GET /chat/sessions/:id`	**200 ✓**
15	Log a study session	`POST /dashboard/study-log`	**201 ✓**
16	Get dashboard summary	`GET /dashboard`	**200 ✓**
17	List subject catalog	`GET /subjects`	**200 ✓**
18	Request nonexistent task → not found	`GET /tasks/:badId`	**404 ✓**
19	Access /auth/me with invalid JWT	`GET /auth/me`	**401 ✓**
20	Request unknown route	`GET /this-route-does-not-exist`	**404 ✓**

Result: 20/20 test cases passed (100%). Full request/response payloads are stored under backend/api_test_results/ for grader review.

8. SCREENSHOTS

The following screenshots are included as separate image files alongside this report (see backend/screenshots/png/):

- 01_database_tables.png — full database inspector output showing all 11 tables, row counts, and keys
- 02_users_table_data.png — live SELECT query result from the users table
- 03_api_test_login.png — API client view of a successful POST /api/auth/login request
- 04_api_test_ai_chat.png — API client view of a successful POST /api/chat/.../messages request
- 05_auth_workflow.png — visual diagram of the registration/login and protected-route flows
- 06_folder_structure.png — backend project folder structure in the code editor

9. CURRENT PROGRESS & NEXT STEPS

Deliverable	This Week	Status
Backend project setup (Express, folder structure)	✓	**Done**
Database schema matching Week 4 ER Diagram	✓	**Done**
JWT authentication + bcrypt password hashing	✓	**Done**
Core CRUD APIs (Auth, Users, Tasks, Chat, Dashboard, Subjects)	✓	**Done**
Database connectivity verified (store + retrieve)	✓	**Done**
Centralized error handling for all failure modes	✓	**Done**
Manual API testing (20 test cases)	✓	**Done**
AI service integration (production OpenAI key)	Next week	**Pending**
Study Plans & Recommendations endpoints	Next week	**Pending**
PostgreSQL migration for staging/production	Next week	**Pending**
Frontend integration	Weeks 13-16	**Upcoming**